

Analytics Project:

Finding Your Fit

Predicting the Right Physical Activity for Young Adults Through Behavioral Profiling

Aicha Dieng

✓ Introduction

In Japan, corporate wellness is not optional; it is institutional. Companies are evaluated under a national health management framework that holds employers accountable for the physical wellbeing of their workforce. Organizations whose employees fall below health benchmarks face higher insurance premiums and reputational consequences. As a result, keeping employees active is not just an HR initiative; it is a business imperative.

This project is framed around a Japanese multinational company looking to implement a personalized employee wellness program across its international offices. The challenge is straightforward: not everyone moves the same way, and not everyone wants to. A blanket "go exercise" policy produces low participation and poor results. What works is meeting people where they are, recommending the activity that fits their body, their lifestyle, and their habits.

This is what this model does. Given an employee's basic health metrics and behavioral patterns, it identifies what type of mover they are and recommends the physical activity: Running, Swimming, or Weight Training is most suited to their profile. The goal is not to force participation, but to lower the barrier to it by making the first recommendation feel personal and achievable.

Why This Population

The analysis focuses on adults aged 18 to 30. This is the age group most likely to be entering the workforce for the first time, and statistically the window where long-term exercise habits are either formed or abandoned. Getting this group moving consistently and in the right direction has compounding benefits over time.

Data

The dataset used is FitLife360, sourced from Kaggle. It is a synthetic health and fitness tracking dataset simulating one year of activity data across 3,000 participants. Each row represents a single workout session and captures demographic information, workout metrics, lifestyle indicators, and health vitals.

The full dataset contains 687,701 sessions. For this analysis, the data was filtered to adults aged 18 to 30 who engage in one of three activities: **Running, Swimming, or Weight Training**. This produced a working subset of 52,557 sessions across the target population.

Analytical Approach

The project combines two methods. First, unsupervised clustering (KMeans) is used to discover natural behavioral profiles among the participants without any predefined labels. Three profiles emerged from this step.

Second, supervised classification (Decision Tree) is used to predict which activity type best matches a person's profile, using the cluster label as one of the input features.

This two-step approach reflects a real intake process: first understand who the person is, then make a recommendation based on that understanding.

- Kaggle Link : <https://www.kaggle.com/datasets/jijagallery/fitlife-health-and-fitness-tracking-dataset>

```
import kagglehub

path = kagglehub.dataset_download("jijagallery/fitlife-health-and-fitness-tracking-dataset")

print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'fitlife-health-and-fitness-tracking-dataset' dataset.
Path to dataset files: /kaggle/input/fitlife-health-and-fitness-tracking-dataset

```
import pandas as pd
import os

print(os.listdir(path))

['health_fitness_dataset.csv']
```

```
df = pd.read_csv(path + "/health_fitness_dataset.csv")
print(df.shape)
df.head()
```

(687701, 22)

	participant_id	date	age	gender	height_cm	weight_kg	activity_type
0	1	2024-01-01	56	F	165.3	53.7	Dancing
1	1	2024-01-04	56	F	165.3	53.9	Swimming
2	1	2024-01-05	56	F	165.3	54.2	Swimming
3	1	2024-01-07	56	F	165.3	54.4	Weight Training
4	1	2024-01-09	56	F	165.3	54.7	Swimming

5 rows × 22 columns

✓ Data Preparation

```
df.isna().sum()
```

	0
participant_id	0
date	0
age	0
gender	0
height_cm	0
weight_kg	0
activity_type	0
duration_minutes	0
intensity	0
calories_burned	0
avg_heart_rate	0
hours_sleep	0
stress_level	0
daily_steps	0
hydration_level	0
bmi	0
resting_heart_rate	0
blood_pressure_systolic	0
blood_pressure_diastolic	0
health_condition	490275
smoking_status	0
fitness_level	0

dtype: int64

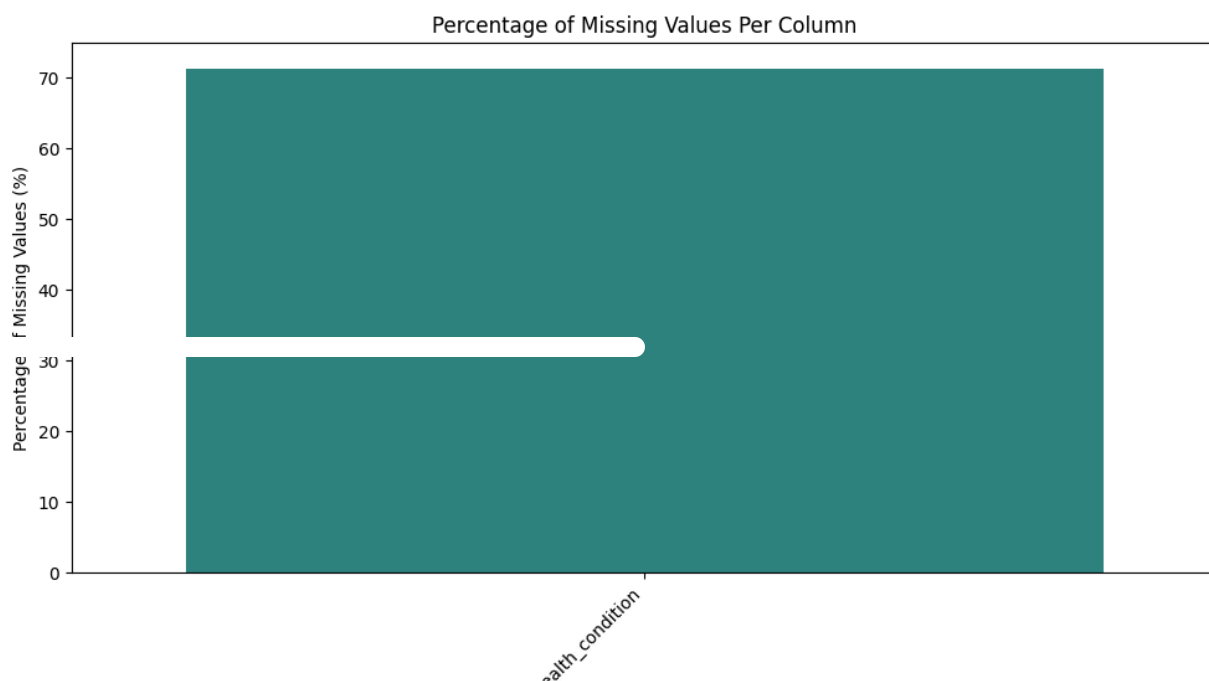
```
import matplotlib.pyplot as plt
import seaborn as sns

missing_percentage = df.isnull().sum() * 100 / len(df)

missing_percentage = missing_percentage[missing_percentage > 0].sort_val

if not missing_percentage.empty:
```

```
plt.figure(figsize=(10, 6))
sns.barplot(x=missing_percentage.index, y=missing_percentage.values,
plt.title('Percentage of Missing Values Per Column')
plt.xlabel('Columns')
plt.ylabel('Percentage of Missing Values (%)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
else:
    print("No missing values found in the dataset.")
```



- Missing values 70% for health condition ... drop the columns

```
df = df.drop('health_condition', axis=1)
print("DataFrame shape after dropping 'health_condition':", df.shape)
df.head()
```

```
DataFrame shape after dropping 'health_condition': (687701, 21)
  participant_id  date  age  gender  height_cm  weight_kg  activity_type
0              1  2024-  56      F      165.3      53.7      Dancing
1              1  2024-  56      F      165.3      53.9      Swimming
2              1  2024-  56      F      165.3      54.2      Swimming
3              1  2024-  56      F      165.3      54.4  Weight Training
4              1  2024-  56      F      165.3      54.7      Swimming

5 rows x 21 columns
```

```
df.tail()

  participant_id  date  age  gender  height_cm  weight_kg  activity_t
687696         3000  2024-  38      F      165.7      112.9      Baske
687697         3000  2024-  38      F      165.7      113.1      Baske
687698         3000  2024-  38      F      165.7      113.4      \
687699         3000  2024-  38      F      165.7      113.6      Baske
687700         3000  2024-  38      F      165.7      113.9      \

5 rows x 21 columns
```

```
print("Column Information (Data Types and Non-Null Counts):")
df.info()

print("\nDescriptive Statistics for Numerical Columns:")
df.describe()
```

Column Information (Data Types and Non-Null Counts):

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 687701 entries, 0 to 687700

Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	participant_id	687701 non-null	int64
1	date	687701 non-null	object
2	age	687701 non-null	int64
3	gender	687701 non-null	object
4	height_cm	687701 non-null	float64
5	weight_kg	687701 non-null	float64
6	activity_type	687701 non-null	object
7	duration_minutes	687701 non-null	int64
8	intensity	687701 non-null	object
9	calories_burned	687701 non-null	float64
10	avg_heart_rate	687701 non-null	int64
11	hours_sleep	687701 non-null	float64
12	stress_level	687701 non-null	int64
13	daily_steps	687701 non-null	int64
14	hydration_level	687701 non-null	float64
15	bmi	687701 non-null	float64
16	resting_heart_rate	687701 non-null	float64
17	blood_pressure_systolic	687701 non-null	float64
18	blood_pressure_diastolic	687701 non-null	float64
19	smoking_status	687701 non-null	object
20	fitness_level	687701 non-null	float64

dtypes: float64(10), int64(6), object(5)

memory usage: 110.2+ MB

Descriptive Statistics for Numerical Columns:

	participant_id	age	height_cm	weight_kg	duration_mi
count	687701.000000	687701.000000	687701.000000	687701.000000	687701.000000
mean	1499.781828	41.658602	168.587699	94.921981	70.000000
std	865.997215	13.581770	9.140811	22.461801	29.000000
min	1.000000	18.000000	145.000000	45.300000	20.000000
25%	749.000000	30.000000	161.700000	78.200000	45.000000
50%	1499.000000	42.000000	168.200000	94.600000	70.000000
75%	2249.000000	53.000000	175.300000	110.500000	95.000000
max	3000.000000	64.000000	198.500000	188.400000	120.000000

df['participant_id'].nunique()

3000

```
print(df['participant_id'].unique())
```

```
[ 1    2    3 ... 2998 2999 3000]
```

```
print(df['age'])
```

```
0      56
1      56
2      56
3      56
4      56
```

```
..
687696   38
687697   38
687698   38
687699   38
687700   38
```

```
Name: age, Length: 687701, dtype: int64
```

```
min_age = df['age'].min()
max_age = df['age'].max()
print(f"Minimum age: {min_age}")
print(f"Maximum age: {max_age}")
```

```
Minimum age: 18
Maximum age: 64
```

```
df_young = df[
    (df['age'] >= 18) &
    (df['age'] <= 30) &
    (df['activity_type'].isin(['Swimming', 'Weight Training', 'Running'])
].copy()

print(df_young.shape)
df_young.head()
```

(52557, 21)

	participant_id	date	age	gender	height_cm	weight_kg	activity_type
482	3	2024-01-09	23	F	158.4	66.7	Weight Training
485	3	2024-01-16	23	F	158.4	67.5	Weight Training
486	3	2024-01-18	23	F	158.4	67.7	Swimming
488	3	2024-01-20	23	F	158.4	68.2	Weight Training
490	3	2024-01-22	23	F	158.4	68.8	Weight Training

5 rows × 21 columns

- Subset with people btw 18-30 that Swim, weight train or/and run

```
df_young['participant_id'].nunique()
```

766

```
df_young.shape
```

(52557, 21)

```
df_young.info()
```

<class 'pandas.core.frame.DataFrame'>
Index: 52557 entries, 482 to 687209
Data columns (total 21 columns):
Column Non-Null Count Dtype
--- -
0 participant_id 52557 non-null int64
1 date 52557 non-null object
2 age 52557 non-null int64
3 gender 52557 non-null object
4 height_cm 52557 non-null float64
5 weight_kg 52557 non-null float64
6 activity_type 52557 non-null object
7 duration_minutes 52557 non-null int64
8 intensity 52557 non-null object
9 calories_burned 52557 non-null float64
10 avg_heart_rate 52557 non-null int64
11 hours_sleep 52557 non-null float64
12 stress_level 52557 non-null int64
13 daily_steps 52557 non-null int64

```

14  hydration_level          52557 non-null float64
15  bmi                     52557 non-null float64
16  resting_heart_rate      52557 non-null float64
17  blood_pressure_systolic 52557 non-null float64
18  blood_pressure_diastolic 52557 non-null float64
19  smoking_status          52557 non-null object
20  fitness_level           52557 non-null float64
dtypes: float64(10), int64(6), object(5)
memory usage: 8.8+ MB

```

```
# Hot encode activity types
```

```
print(df_young['intensity'].unique())
```

```
['High' 'Low' 'Medium']
```

```

numerical_cols = df.select_dtypes(include=['number']).columns.tolist()
categorical_cols = df.select_dtypes(include=['object', 'category']).columns.tolist()

```

```

print("Numerical Columns:", numerical_cols)
print("Categorical Columns:", categorical_cols)

```

```

Numerical Columns: ['participant_id', 'age', 'height_cm', 'weight_kg', 'duration_minutes', 'intensity', 'calories_burned', 'avg_heart_rate', 'hours_sleep', 'stress_level', 'daily_steps', 'hydration_level', 'bmi', 'resting_heart_rate', 'blood_pressure_systolic', 'blood_pressure_diastolic', 'smoking_status', 'fitness_level']
Categorical Columns: ['date', 'gender', 'activity_type']

```

```
df_young['date'] = pd.to_datetime(df_young['date'])
```

```
print(df_young.dtypes)
```

```

participant_id          int64
date                   datetime64[ns]
age                    int64
gender                 object
height_cm              float64
weight_kg              float64
activity_type          object
duration_minutes       int64
intensity              object
calories_burned        float64
avg_heart_rate         int64
hours_sleep            float64
stress_level           int64
daily_steps            int64
hydration_level        float64
bmi                   float64
resting_heart_rate     float64
blood_pressure_systolic float64
blood_pressure_diastolic float64
smoking_status         object
fitness_level          float64
dtype: object

```

Before any analysis, the dataset was reviewed for completeness and consistency. Out of 22 columns, only one had a significant missing value issue: the health condition column, which was missing for roughly 70% of participants. Since the column was mostly empty, it was removed from the dataset entirely.

A few columns also needed formatting adjustments before the analysis could proceed. Dates were stored as plain text and were converted into a proper date format to allow time-based analysis. Intensity levels, originally recorded as Low, Medium, and High, were converted to numerical values to reflect their natural order and make them usable in the model.

All other columns covering demographics, workout metrics, lifestyle indicators, and health vitals were already in the right format and required no changes.

✓ Exploratory Data Analysis (EDA)

```
print("Descriptive Statistics for Numerical Columns:")
df_young.describe()
```

Statistics for Numerical Columns:					
	participant_id	date	age	height_cm	weight_kg
	2557.000000	52557	52557.000000	52557.000000	52557.000000
	1535.510836	2024-06-16 10:34:31.762087168	23.814335	168.503166	94.009900
	3.000000	2024-01-01 00:00:00	18.000000	145.000000	45.300000
	786.000000	2024-03-21 00:00:00	21.000000	161.600000	77.200000
	1531.000000	2024-06-06 00:00:00	24.000000	168.100000	93.400000
	2301.000000	2024-09-10 00:00:00	27.000000	175.300000	109.400000
	2998.000000	2024-12-25 00:00:00	30.000000	198.500000	172.200000
	872.547877	NaN	3.743072	9.496855	22.771000

```
print("Descriptive Statistics for Categorical Columns:")
for col in categorical_cols:
```

```
print(f"\n--- {col} ---")
print(df_young[col].value_counts())
```

Descriptive Statistics for Categorical Columns:

```
--- date ---
date
2024-04-20    201
2024-04-22    199
2024-03-20    196
2024-04-24    194
2024-04-17    193
...
2024-10-31    103
2024-10-23    102
2024-10-03    101
2024-08-31    100
2024-09-15     97
Name: count, Length: 360, dtype: int64
```

```
--- gender ---
gender
M      26099
F      25338
Other   1120
Name: count, dtype: int64
```

```
--- activity_type ---
activity_type
Weight Training    18055
Swimming           17517
Running            16985
Name: count, dtype: int64
```

```
--- intensity ---
intensity
Medium    26266
Low       15756
High      10535
Name: count, dtype: int64
```

```
--- smoking_status ---
smoking_status
Never      32951
Former     12067
Current     7539
Name: count, dtype: int64
```

```
print("Mode for each column:")
for col in df_young.columns:
    mode_val = df_young[col].mode().tolist()
    print(f"Column '{col}': {mode_val}")
```

```

Mode for each column:
Column 'participant_id': [1880, 2155]
Column 'date': [Timestamp('2024-04-20 00:00:00')]
Column 'age': [21]
Column 'gender': ['M']
Column 'height_cm': [170.9]
Column 'weight_kg': [94.6]
Column 'activity_type': ['Weight Training']
Column 'duration_minutes': [53]
Column 'intensity': ['Medium']
Column 'calories_burned': [8.7]
Column 'avg_heart_rate': [147]
Column 'hours_sleep': [7.0]
Column 'stress_level': [5]
Column 'daily_steps': [9500]
Column 'hydration_level': [2.7]
Column 'bmi': [21.4]
Column 'resting_heart_rate': [73.3]
Column 'blood_pressure_systolic': [122.6]
Column 'blood_pressure_diastolic': [82.9]
Column 'smoking_status': ['Never']
Column 'fitness_level': [9.62]

```

```

import matplotlib.pyplot as plt
import seaborn as sns

# Identify numerical columns in df_young
numerical_cols_df_young = df_young.select_dtypes(include=['number']).col

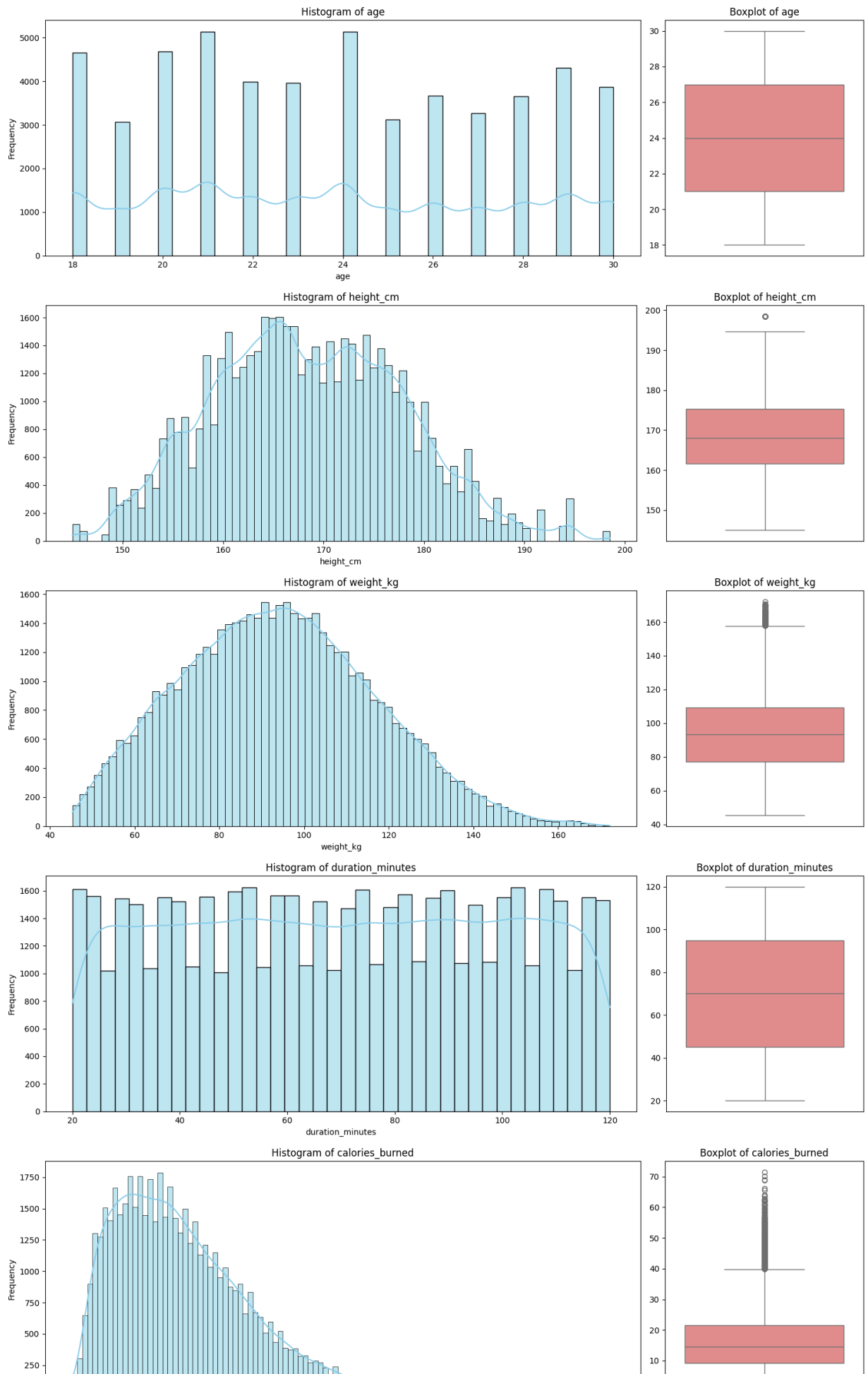
# Plot histogram and boxplot for each numerical column
for col in numerical_cols_df_young:
    if col != 'participant_id': # participant_id is an identifier, not a
        fig, axes = plt.subplots(1, 2, figsize=(15, 5), gridspec_kw={'wi

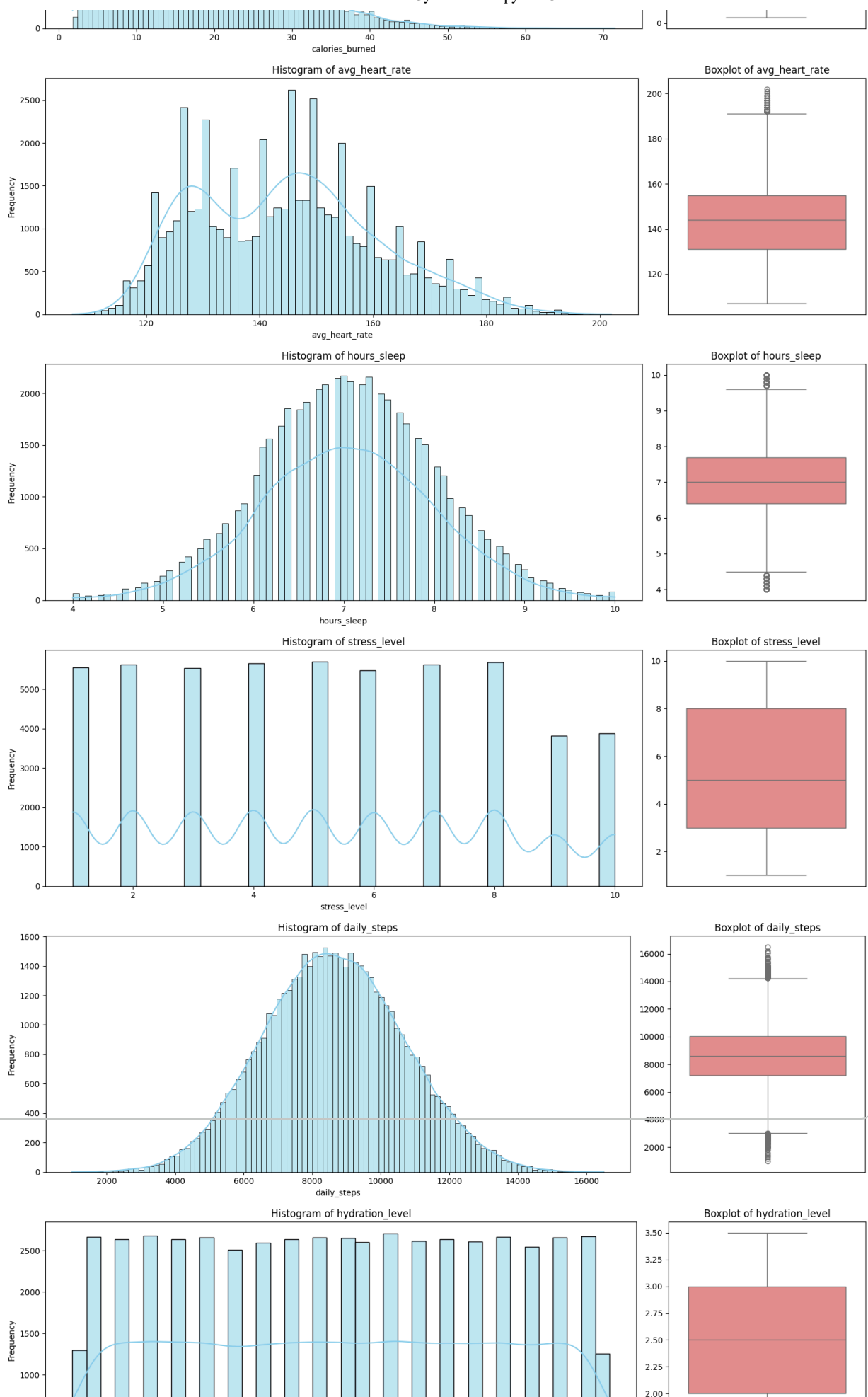
        # Histogram
        sns.histplot(df_young[col], kde=True, ax=axes[0], color='skyblue
        axes[0].set_title(f'Histogram of {col}')
        axes[0].set_xlabel(col)
        axes[0].set_ylabel('Frequency')

        # Boxplot
        sns.boxplot(y=df_young[col], ax=axes[1], color='lightcoral')
        axes[1].set_title(f'Boxplot of {col}')
        axes[1].set_ylabel('') # Remove y-label for boxplot as it's redu
        axes[1].set_xticks([]) # Remove x-axis ticks for boxplot

    plt.tight_layout()
    plt.show()

```



Descriptive statistics were calculated for both numerical and categorical columns to get a sense of the range, distribution, and typical values across the dataset.

- The average age in the subset is 24, which aligns with the 18 to 30 focus.
- Weight ranges widely from 45kg to 172kg reflecting genuine variation in body composition across participants.
- Fitness level is fairly spread out, suggesting the subset captures people at different stages of their fitness journey rather than a uniform group.

On the categorical side, the three activities are reasonably balanced

- Weight training leads slightly with 18,055 sessions, followed by Swimming at 17,517 and Running at 16,985.

- Intensity is skewed toward Medium, which is the most common level across sessions. Most participants have never smoked, and gender is fairly split between male and female with a small Other category.

Histograms and boxplots were generated for every numerical column to visualize the distributions.

- Most health indicators, resting heart rate, blood pressure, and sleep follow roughly normal distributions with no extreme outliers.
- Stress level appears more uniform, suggesting it is evenly spread across the population.
- Calories burned are right-skewed, meaning most sessions burn a moderate amount with a smaller number of high-output sessions pulling the distribution upward.

Overall, the data has no extreme outliers that would distort the analysis, and a reasonable spread across all key variables.

✓ Feature Engineering

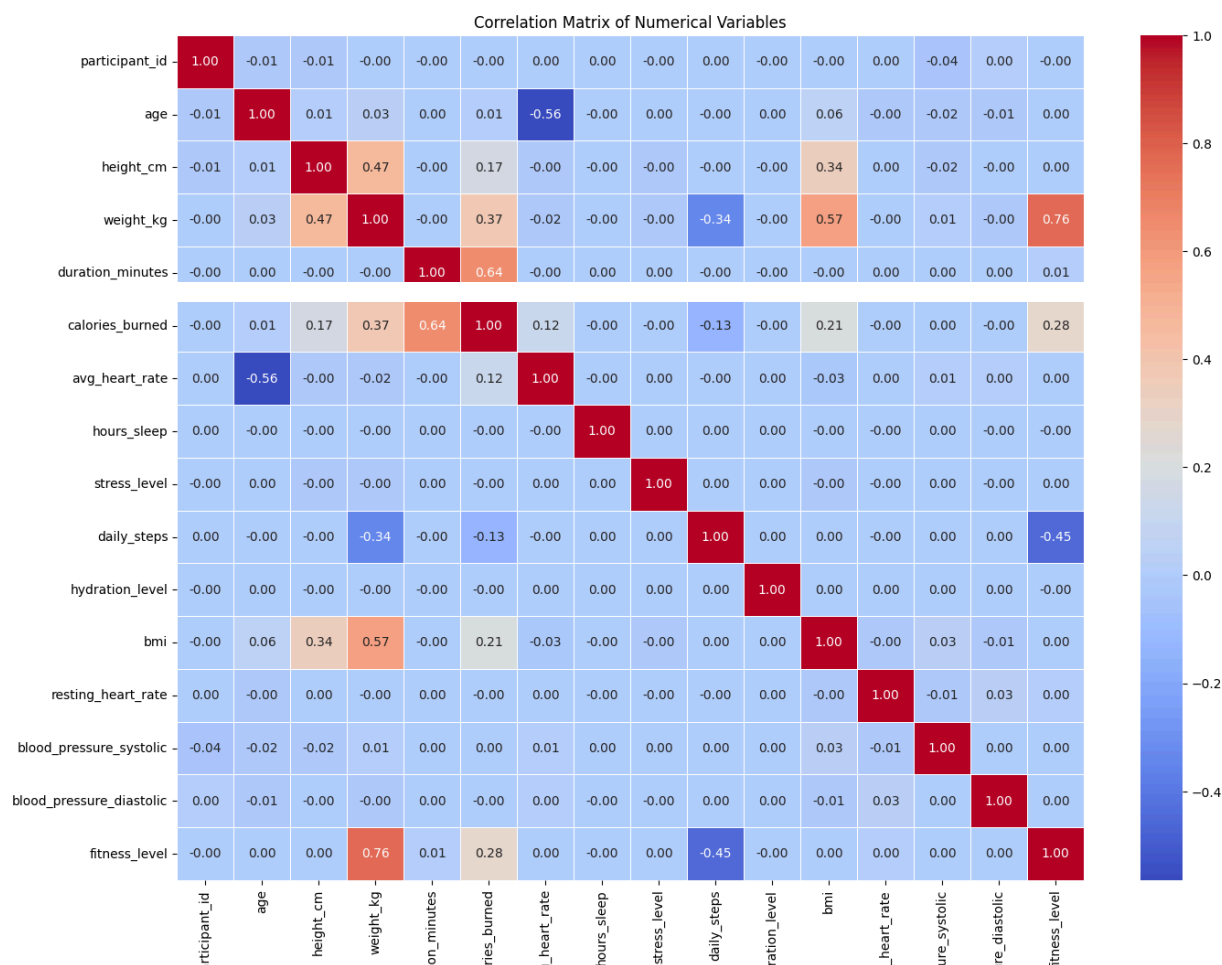
✓ Interaction terms

```
import matplotlib.pyplot as plt
import seaborn as sns

# Select only numerical columns from the original DataFrame df
numerical_df = df.select_dtypes(include=['number'])

# Calculate the correlation matrix
corr_matrix = numerical_df.corr()
```

```
# Plotting the correlation matrix as a heatmap
plt.figure(figsize=(16, 12))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidth
plt.title('Correlation Matrix of Numerical Variables')
plt.show()
```



```
import numpy as np
```

```
# Get the correlation matrix from the kernel state (assuming 'corr_matrix' is available)
# If 'corr_matrix' is not available, uncomment and run the previous cell
# numerical_df = df.select_dtypes(include=['number'])
# corr_matrix = numerical_df.corr()
```

```
# Flatten the correlation matrix and remove self-correlations
correlations = corr_matrix.stack()
```

```
# Filter out self-correlations and duplicate pairs
# Keep only one side of the symmetric matrix (e.g., A-B but not B-A)
filtered_correlations = correlations[correlations.index.get_level_values(0) < correlations.index.get_level_values(1)]
filtered_correlations = filtered_correlations.loc[~filtered_correlations.index.duplicated(keep='first')]
```

```
# Sort the correlations
sorted_correlations = filtered_correlations.sort_values(ascending=False)
```

```
print("Top 5 positive correlations:")
print(sorted_correlations.head(5))

print("\nTop 5 negative correlations:")
print(sorted_correlations.tail(5))
```

Top 5 positive correlations:

weight_kg	fitness_level	0.761940
fitness_level	weight_kg	0.761940
calories_burned	duration_minutes	0.641077
duration_minutes	calories_burned	0.641077
bmi	weight_kg	0.573853

dtype: float64

Top 5 negative correlations:

daily_steps	weight_kg	-0.343977
	fitness_level	-0.449572
fitness_level	daily_steps	-0.449572
avg_heart_rate	age	-0.564767
age	avg_heart_rate	-0.564767

dtype: float64

```
df_model = df_young.copy()

df_model['weight_fitness'] = df_model['weight_kg'] * df_model['fitness_level']
df_model['calories_duration'] = df_model['calories_burned'] * df_model['duration_minutes']
df_model['bmi_weight'] = df_model['bmi'] * df_model['weight_kg']

print(df_model.shape)
df_model.head()
```

(52557, 24)

	participant_id	date	age	gender	height_cm	weight_kg	activity_type
482	3	2024-01-09	23	F	158.4	66.7	Weight Training
485	3	2024-01-16	23	F	158.4	67.5	Weight Training
486	3	2024-01-18	23	F	158.4	67.7	Swimming
488	3	2024-01-20	23	F	158.4	68.2	Weight Training
490	3	2024-01-22	23	F	158.4	68.8	Weight Training

5 rows x 24 columns

```
df_model.head()
```

participant_id	date	age	gender	height_cm	weight_kg	activity_type
482	3 2024-01-09	23	F	158.4	66.7	Weight Training
485	3 2024-01-16	23	F	158.4	67.5	Weight Training
486	3 2024-01-18	23	F	158.4	67.7	Swimming
488	3 2024-01-20	23	F	158.4	68.2	Weight Training
490	3 2024-01-22	23	F	158.4	68.8	Weight Training

5 rows × 24 columns

df_model[['weight_fitness', 'calories_duration', 'bmi_weight']].describe

	weight_fitness	calories_duration	bmi_weight	
count	52557.000000	52557.000000	52557.000000	
mean	988.193322	1337.379618	2156.677624	
std	710.885855	1120.381324	783.206167	
min	1.380000	36.000000	715.740000	
25%	369.250000	431.200000	1577.180000	
50%	869.526000	1046.500000	2033.520000	
75%	1525.188000	1966.200000	2606.340000	
max	3437.280000	8142.000000	5831.000000	

```
df_model[['weight_fitness', 'calories_duration', 'bmi_weight',  
          'weight_kg', 'fitness_level', 'calories_burned',  
          'duration_minutes', 'bmi']].corr().round(2)
```

	weight_fitness	calories_duration	bmi_weight	weight_kg
weight_fitness	1.00	0.25	0.67	0.87
calories_duration	0.25	1.00	0.26	0.28
bmi_weight	0.67	0.26	1.00	0.93
weight_kg	0.87	0.28	0.93	1.00
fitness_level	0.96	0.22	0.51	0.77
calories_burned	0.37	0.96	0.39	0.43
duration_minutes	-0.00	0.84	-0.00	-0.01
bmi	0.20	0.17	0.83	0.58

- $\text{weight_fitness} = \text{weight_kg} \times \text{fitness_level}$
 - $\text{calories_duration} = \text{calories_burned} \times \text{duration_minutes}$
 - $\text{bmi_weight} = \text{bmi} \times \text{weight_kg}$
-
- $\text{weight_fitness} = \text{weight_kg} \times \text{fitness_level}$ (Fitness level has 0.96, so drop)
 - $\text{calories_duration} = \text{calories_burned} \times \text{duration_minutes}$ (cal burned 0.96)
 - $\text{bmi_weight} = \text{bmi} \times \text{weight_kg}$ (bmi is litch 1 and weigth is 93) so the interactions terms are useless

```
df_model = df_model.drop(columns=['weight_fitness', 'bmi_weight', 'calor
print(df_model.shape)
```

```
(52557, 21)
```

```
# encode intensity first
df_model['intensity'] = df_model['intensity'].map({'Low': 0, 'Medium': 1

# create new interaction terms
df_model['stress_sleep'] = df_model['stress_level'] * df_model['hours_sl
df_model['effort_score'] = df_model['duration_minutes'] * df_model['inte
df_model['cardio_effort'] = df_model['calories_burned'] * df_model['avg_

# evaluate them
df_model[['stress_sleep', 'effort_score', 'cardio_effort',
          'stress_level', 'hours_sleep', 'duration_minutes',
          'intensity', 'calories_burned', 'avg_heart_rate']].corr().roun
```

	stress_sleep	effort_score	cardio_effort	stress_level	h
stress_sleep	1.00	-0.01	-0.01	0.96	
effort_score	-0.01	1.00	0.65	-0.01	
cardio_effort	-0.01	0.65	1.00	-0.01	
stress_level	0.96	-0.01	-0.01	1.00	
hours_sleep	0.24	-0.00	-0.00	-0.01	
duration_minutes	-0.01	0.44	0.69	-0.01	
intensity	-0.00	0.83	0.36	-0.00	
calories_burned	-0.01	0.53	0.98	-0.01	
avg_heart_rate	-0.00	0.72	0.36	-0.00	

- Stress lv * h sleep = 0.96 for stress
- effort score and duration min = 0.44 for duration min and 0.83 for intensity ... interesting
- calories effort= cal burned = 0.98 , heart rate : 0.36

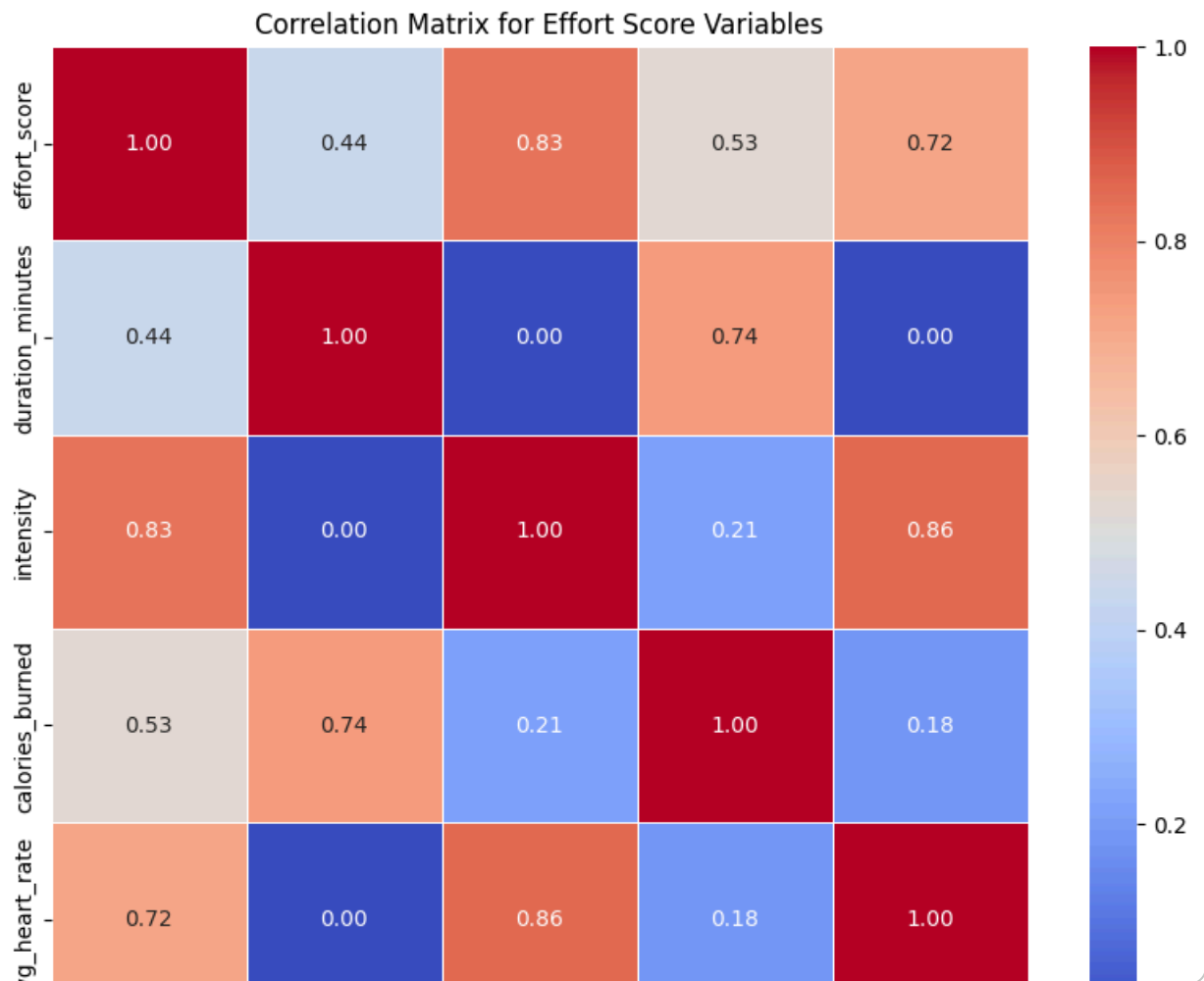
```
df_model = df_model.drop(columns=['stress_sleep', 'cardio_effort'])
print(df_model.shape)
print(df_model.columns.tolist())
```

```
(52557, 22)
['participant_id', 'date', 'age', 'gender', 'height_cm', 'weight_kg', 'ac
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate the correlation matrix for effort_score and related variable
corr_effort = df_model[['effort_score', 'duration_minutes', 'intensity',

# Plotting the correlation matrix as a heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(corr_effort, annot=True, cmap='coolwarm', fmt=".2f", linewidth
plt.title('Correlation Matrix for Effort Score Variables')
plt.show()
```



```
df_model['cal_duration'] = df_model['calories_burned'] * df_model['duration_minutes']
df_model[['cal_duration', 'calories_burned', 'duration_minutes']].corr()
```

	cal_duration	calories_burned	duration_minutes
cal_duration	1.00	0.96	0.84
calories_burned	0.96	1.00	0.74
duration_minutes	0.84	0.74	1.00

- Cal burned has a too corr

```
df_model['heart_effort'] = df_model['avg_heart_rate'] * df_model['effort_score']
df_model[['heart_effort', 'avg_heart_rate', 'effort_score']].corr().round(2)
```

	heart_effort	avg_heart_rate	effort_score	
heart_effort	1.00	0.75	0.99	
avg_heart_rate	0.75	1.00	0.72	
effort_score	0.99	0.72	1.00	

- effort score is 0.99 so no

```
df_model = df_model.drop(columns=['cal_duration', 'heart_effort'])
print(df_model.columns.tolist())
```

```
['participant_id', 'date', 'age', 'gender', 'height_cm', 'weight_kg', 'ac
```

```
print("Kept interaction term: 'effort_score'")
```

```
Kept interaction term: 'effort_score'
```

The correlation matrix revealed several strong relationships between variables.

Three combinations were tried first, based on the strongest correlations in the data:

- weight × fitness level (0.76),
- calories burned × duration (0.64), and
- BMI × weight (0.57).

The reasoning was that variables already moving together might produce a stronger combined signal. That turned out to be the wrong approach; all three correlated above 0.90 with at least one original variable and were dropped.

A second test was used with more independent variables. The rule applied was simple: if an interaction term correlates above 0.90 with any of its original variables, it is redundant and gets dropped.

- Stress × sleep correlated 0.96 with stress level — dropped.
- Calories × heart rate correlated 0.98 with calories burned — dropped.
- Duration × intensity correlated 0.44 with duration and 0.83 with intensity, both under the threshold — kept.

The one that was maintained was the effort score. A person training 90 minutes at high intensity is fundamentally different from someone training 90 minutes at low intensity. Effort score captures that difference in a single number, which neither variable could do alone.

- $\text{effort_score} = (\text{duration_minutes} \times \text{intensity})$.

✓ Clustering

```
print(df_model.columns.tolist())  
print(df_model.shape)
```

```
['participant_id', 'date', 'age', 'gender', 'height_cm', 'weight_kg', 'ac  
(52557, 22)
```

```
# drop non-numerical and identifier columns  
cluster_df = df_model.drop(columns=['participant_id', 'date', 'activity_  
  
print(cluster_df.shape)  
print(cluster_df.columns.tolist())
```

```
(52557, 17)  
['age', 'height_cm', 'weight_kg', 'duration_minutes', 'intensity', 'calor
```

✓ Timeline breakdown

```
# extract from date column in df_young (still has date)  
date_features = df_young.groupby('participant_id').agg(  
    total_active_days=('date', 'nunique'),  
    active_months=('date', lambda x: x.dt.month.nunique())  
) .reset_index()  
  
print(date_features.shape)  
date_features.head(10)
```

(766, 3)

	participant_id	total_active_days	active_months	
0	3	45	12	
1	7	67	11	
2	9	66	10	
3	12	73	11	
4	17	53	12	
5	25	50	9	
6	26	78	12	
7	28	45	11	
8	34	65	10	
9	35	68	12	

Étapes suivantes :

[Générer du code avec date_features](#)[New interactive sheet](#)

```
print(date_features['participant_id'].nunique())
```

766

```
print(date_features[['total_active_days', 'active_months']].describe())
```

	total_active_days	active_months
count	766.000000	766.000000
mean	68.612272	10.608355
std	17.902271	1.216717
min	22.000000	6.000000
25%	56.000000	10.000000
50%	68.000000	11.000000
75%	81.000000	11.000000
max	132.000000	12.000000

```
date_features = df_young.groupby('participant_id').agg(
    total_active_days=('date', 'nunique'),
    active_months=('date', lambda x: x.dt.month.nunique())
).reset_index()
```

```
# add longest streak
def longest_streak(dates):
    months = sorted(dates.dt.month.unique())
    max_streak = 1
    current = 1
    for i in range(1, len(months)):
```

```

        if months[i] == months[i-1] + 1:
            current += 1
            max_streak = max(max_streak, current)
        else:
            current = 1
    return max_streak

streak = df_young.groupby('participant_id')['date'].apply(longest_streak)
streak.columns = ['participant_id', 'longest_streak']

date_features = date_features.merge(streak, on='participant_id')

print(date_features.shape)
date_features.head()

```

(766, 4)

	participant_id	total_active_days	active_months	longest_streak	
0	3	45	12	12	
1	7	67	11	7	
2	9	66	10	10	
3	12	73	11	9	
4	17	53	12	12	

Étapes suivantes :

[Générer du code avec date_features](#)[New interactive sheet](#)

- Merging all df (injoin one)

```

df_model2 = df_model.copy()
df_model2 = df_model2.merge(date_features, on='participant_id', how='left')

print(df_model2.shape)
print(df_model2.columns.tolist())

```

```

(52557, 25)
['participant_id', 'date', 'age', 'gender', 'height_cm', 'weight_kg', 'ac

```

```

cluster_df = df_model2.drop(columns=['participant_id', 'date', 'activity'])
print(cluster_df.shape)

```

(52557, 20)

The date column carried more information than just when a session happened. By looking across all sessions per person, it was possible to understand how often someone showed up and how consistently they did so over the year.

Three features were extracted from this:

- the total number of unique days a participant was active,
- the number of distinct months they showed up, and
- the length of their longest consecutive streak of active months.

These features were added to the dataset before clustering, on the basis that consistency patterns could help define more meaningful profiles, someone active across 11 months tells a different story than someone who showed up intensely for 2 months and stopped.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
cluster_scaled = scaler.fit_transform(cluster_df)

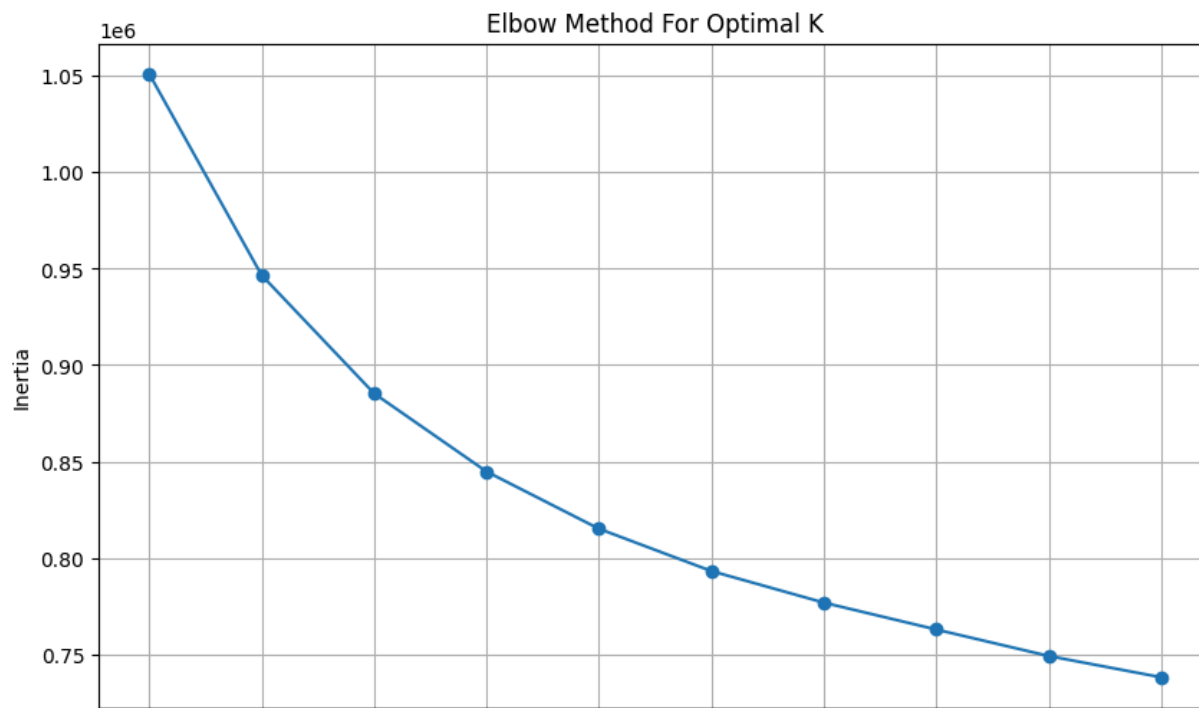
print(cluster_scaled.shape)
```

```
(52557, 20)
```

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertia = []
# Consider a reasonable range for K, e.g., from 1 to 10
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(cluster_scaled)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), inertia, marker='o')
plt.title('Elbow Method For Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()
```



```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# k=3
kmeans3 = KMeans(n_clusters=3, random_state=42, n_init=10)
labels3 = kmeans3.fit_predict(cluster_scaled)
score3 = silhouette_score(cluster_scaled, labels3, sample_size=5000, ran

# k=4
kmeans4 = KMeans(n_clusters=4, random_state=42, n_init=10)
labels4 = kmeans4.fit_predict(cluster_scaled)
score4 = silhouette_score(cluster_scaled, labels4, sample_size=5000, ran

print(f"k=3 silhouette score: {score3:.4f}")
print(f"k=4 silhouette score: {score4:.4f}")
```

```
k=3 silhouette score: 0.0784
k=4 silhouette score: 0.0721
```

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# k=3
kmeans3 = KMeans(n_clusters=3, random_state=42, n_init=10)
labels3 = kmeans3.fit_predict(cluster_scaled)
score3 = silhouette_score(cluster_scaled, labels3, sample_size=5000, ran

# k=4
kmeans4 = KMeans(n_clusters=4, random_state=42, n_init=10)
labels4 = kmeans4.fit_predict(cluster_scaled)
```

```
score4 = silhouette_score(cluster_scaled, labels4, sample_size=5000, ran

print(f"k=3 silhouette score: {score3:.4f}")
print(f"k=4 silhouette score: {score4:.4f}")
```

```
k=3 silhouette score: 0.0784
k=4 silhouette score: 0.0721
```

```
from sklearn.metrics import silhouette_samples
import matplotlib.pyplot as plt
import numpy as np

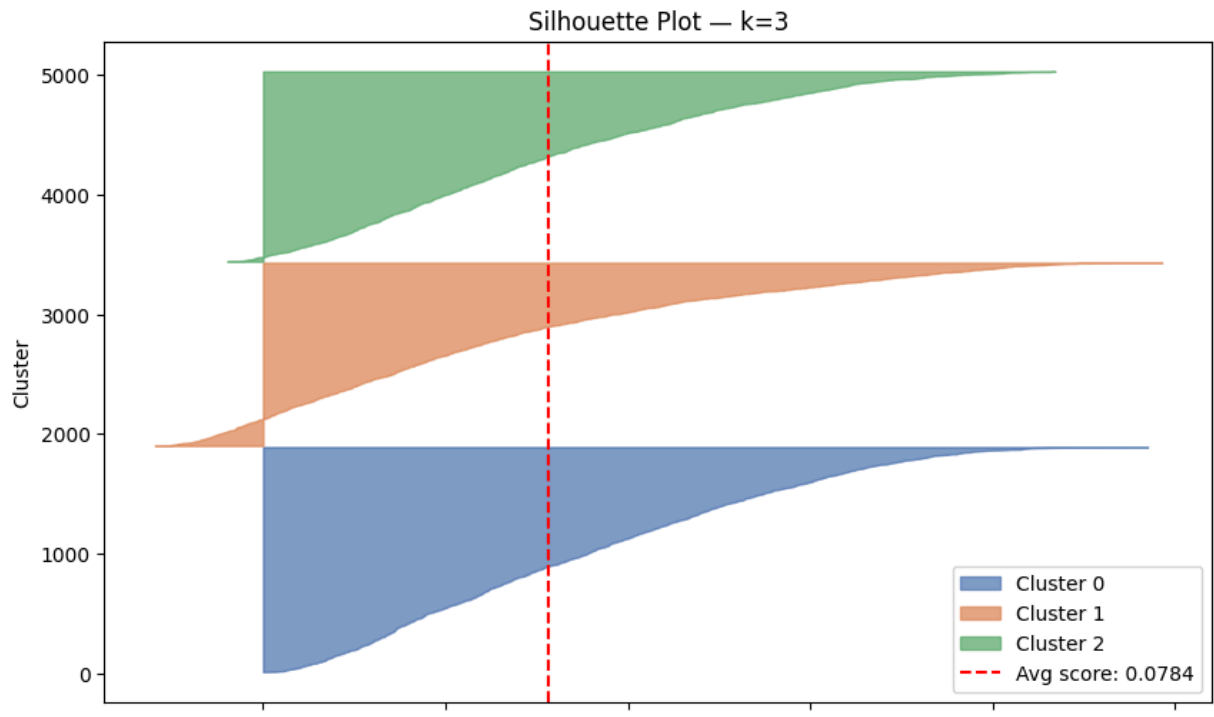
sample_idx = np.random.choice(len(cluster_scaled), 5000, replace=False)
X_sample = cluster_scaled[sample_idx]
labels_sample = labels3[sample_idx]

silhouette_vals = silhouette_samples(X_sample, labels_sample)

fig, ax = plt.subplots(figsize=(10, 6))
y_lower = 10
colors = ['#4C72B0', '#DD8452', '#55A868']

for i in range(3):
    cluster_vals = np.sort(silhouette_vals[labels_sample == i])
    size = cluster_vals.shape[0]
    y_upper = y_lower + size
    ax.fill_betweenx(np.arange(y_lower, y_upper), 0, cluster_vals,
                     alpha=0.7, color=colors[i], label=f'Cluster {i}')
    y_lower = y_upper + 10

ax.axvline(x=score3, color='red', linestyle='--', label=f'Avg score: {sc
ax.set_xlabel('Silhouette Score')
ax.set_ylabel('Cluster')
ax.set_title('Silhouette Plot - k=3')
ax.legend()
plt.show()
```



```

from sklearn.metrics import silhouette_samples
import matplotlib.pyplot as plt
import numpy as np

sample_idx = np.random.choice(len(cluster_scaled), 5000, replace=False)
X_sample = cluster_scaled[sample_idx]
labels_sample = labels4[sample_idx]

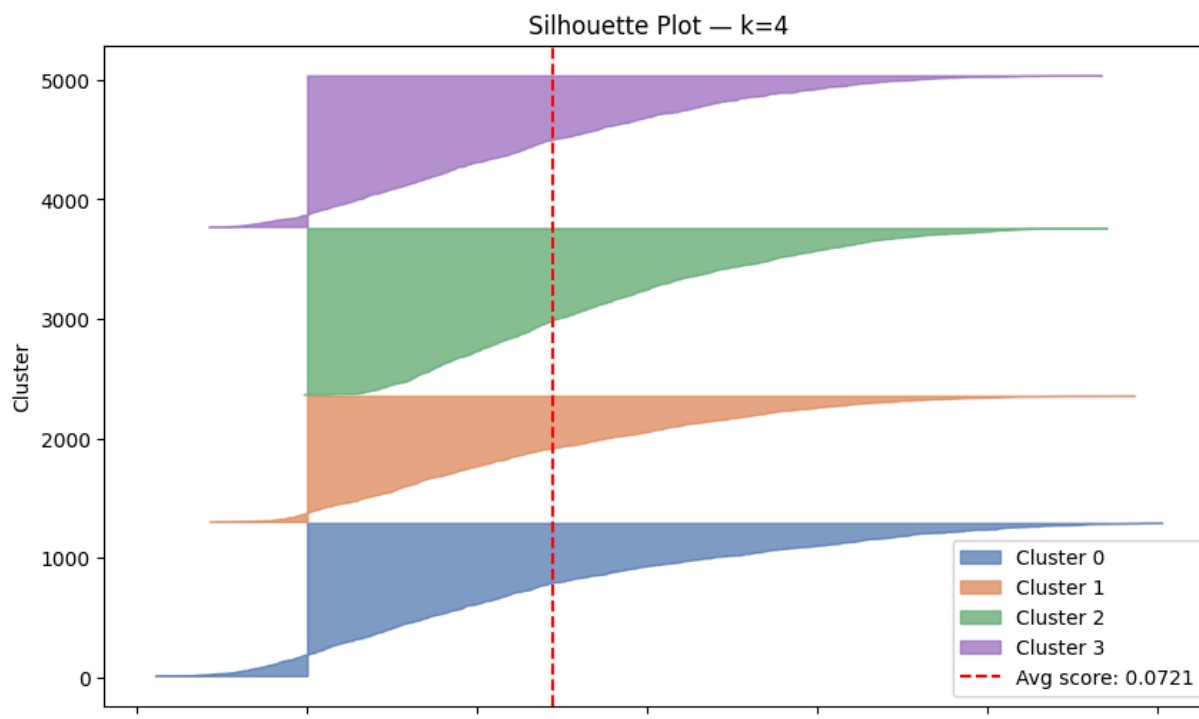
silhouette_vals = silhouette_samples(X_sample, labels_sample)

fig, ax = plt.subplots(figsize=(10, 6))
y_lower = 10
colors = ['#4C72B0', '#DD8452', '#55A868', '#9467BD']

for i in range(4):
    cluster_vals = np.sort(silhouette_vals[labels_sample == i])
    size = cluster_vals.shape[0]
    y_upper = y_lower + size
    ax.fill_betweenx(np.arange(y_lower, y_upper), 0, cluster_vals,
                     alpha=0.7, color=colors[i], label=f'Cluster {i}')
    y_lower = y_upper + 10

ax.axvline(x=score4, color='red', linestyle='--', label=f'Avg score: {sc
ax.set_xlabel('Silhouette Score')
ax.set_ylabel('Cluster')
ax.set_title('Silhouette Plot - k=4')
ax.legend()
plt.show()

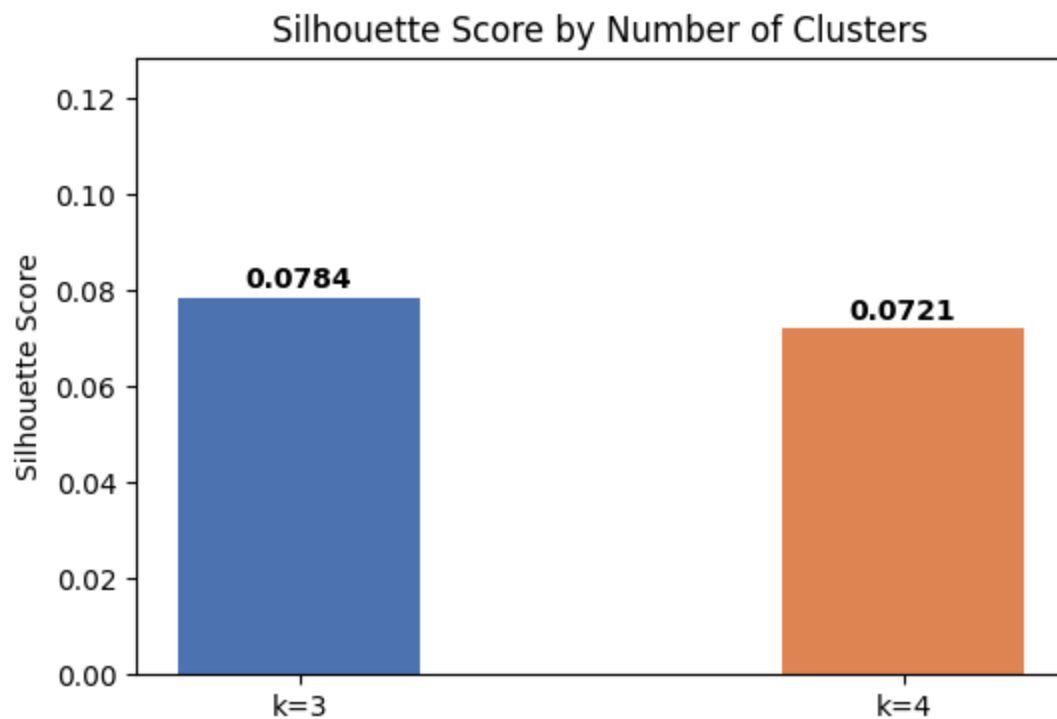
```



```
import matplotlib.pyplot as plt

scores = [score3, score4]
labels = ['k=3', 'k=4']
colors = ['#4C72B0', '#DD8452']

plt.figure(figsize=(6, 4))
bars = plt.bar(labels, scores, color=colors, width=0.4)
plt.title('Silhouette Score by Number of Clusters')
plt.ylabel('Silhouette Score')
plt.ylim(0, max(scores) + 0.05)
for bar, score in zip(bars, scores):
    plt.text(bar.get_x() + bar.get_width()/2,
             bar.get_height() + 0.002,
             f'{score:.4f}', ha='center', fontweight='bold')
plt.show()
```



```
df_model2['cluster'] = labels3
```

```
cluster_profile = df_model2.groupby('cluster')[cluster_df.columns.tolist()  
cluster_profile.T
```

cluster	0	1	2
age	23.91	23.59	23.92
height_cm	164.75	169.85	171.62
weight_kg	74.40	101.15	110.20
duration_minutes	61.39	89.30	61.64
intensity	0.80	1.50	0.44
calories_burned	10.52	24.58	15.02
avg_heart_rate	142.46	157.61	134.92
hours_sleep	7.04	7.05	7.05
stress_level	5.28	5.18	5.25
daily_steps	9619.96	8352.81	7749.08
hydration_level	2.50	2.50	2.49
bmi	21.00	23.03	23.59
resting_heart_rate	70.17	70.10	70.19
blood_pressure_systolic	119.72	120.06	120.06
blood_pressure_diastolic	80.38	80.22	80.46
fitness_level	5.47	10.95	12.83
effort_score	44.59	128.92	21.30
total_active_days	70.50	73.62	76.22
active_months	10.46	10.84	11.07
longest_streak	8.14	8.94	9.51

```

cluster_labels = {
    0: 'Moderate Active - Steps Focused',
    1: 'High-Intensity Training',
    2: 'Consistent Low-Intensity Fitness'
}

# Map the labels to the cluster profile
cluster_profile_labeled = cluster_profile.rename(index=cluster_labels)

print("Cluster Profiles with Labels:")
print(cluster_profile_labeled.T)

# You can also add the labels back to the main DataFrame if needed:

```

```
df_model2['cluster_label'] = df_model2['cluster'].map(cluster_labels)
print(df_model2[['cluster', 'cluster_label']].head())
```

resting_heart_rate	70.17
blood_pressure_systolic	119.72
blood_pressure_diastolic	80.38
fitness_level	5.47
effort_score	44.59
total_active_days	70.50
active_months	10.46
longest_streak	8.14

cluster	High-Intensity Training \
age	23.59
height_cm	169.85
weight_kg	101.15
duration_minutes	89.30
intensity	1.50
calories_burned	24.58
avg_heart_rate	157.61
hours_sleep	7.05
stress_level	5.18
daily_steps	8352.81
hydration_level	2.50
bmi	23.03
resting_heart_rate	70.10
blood_pressure_systolic	120.06
blood_pressure_diastolic	80.22
fitness_level	10.95
effort_score	128.92
total_active_days	73.62
active_months	10.84
longest_streak	8.94

cluster	Consistent Low-Intensity Fitness
age	23.92
height_cm	171.62
weight_kg	110.20
duration_minutes	61.64
intensity	0.44
calories_burned	15.02
avg_heart_rate	134.92

```

0      0 Moderate Active - Steps Focused
1      0 Moderate Active - Steps Focused
2      0 Moderate Active - Steps Focused
3      1 High-Intensity Training
4      0 Moderate Active - Steps Focused

```

```
# Checking weight changes overtime
```

```

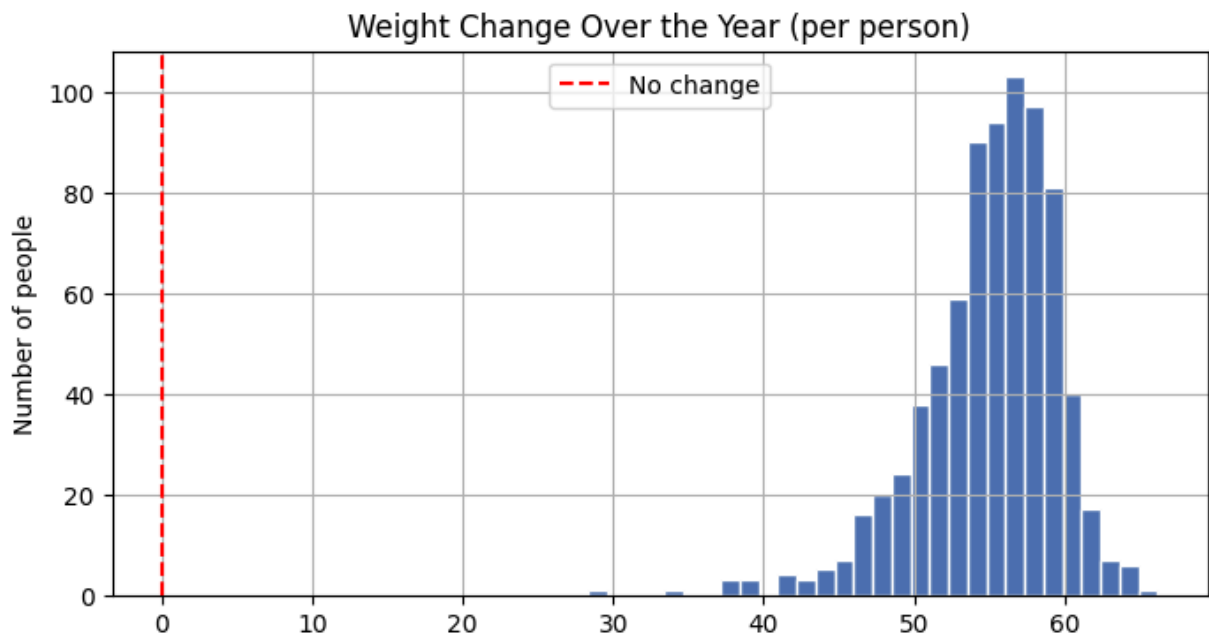
df_young_sorted = df_young.sort_values(['participant_id', 'date'])

weight_change = df_young_sorted.groupby('participant_id').agg(
    first_weight=('weight_kg', 'first'),
    last_weight=('weight_kg', 'last')
).reset_index()

weight_change['weight_change'] = weight_change['last_weight'] - weight_c

weight_change['weight_change'].hist(bins=30, figsize=(8,4), color='#4C72
plt.title('Weight Change Over the Year (per person)')
plt.xlabel('Weight change (kg)')
plt.ylabel('Number of people')
plt.axvline(x=0, color='red', linestyle='--', label='No change')
plt.legend()
plt.show()

```



- Weight change over time was explored out of curiosity to see whether any patterns emerged across the year. The findings showed that all participants gained weight over the course of the year with no variation in the other direction. Given this uniform pattern, no meaningful insight could be drawn from it and it was not incorporated into the model.

Clustering Summary:

K-Means was chosen over hierarchical clustering for practical reasons: with 52,557 sessions in the dataset, hierarchical clustering would be too slow. KMeans is faster and well suited to this scale.

Before running the algorithm,

- All numerical columns were scaled so that no single variable, like daily steps ranging in the thousands, would dominate over variables measured on a smaller scale like hydration level.
- Non-numerical columns such as participant ID, date, activity type, gender, and smoking status were excluded, as the model can only work with numbers.

Choosing the number of clusters

KMeans requires deciding upfront how many groups to look for.

The elbow method plots curve showed no sharp bend, the data did not naturally point to one An obvious number of clusters.

K=3 and K=4 were compared directly using the silhouette score.

K=3 scored 0.0784 against k=4 at 0.0721. The difference is small, but k=3 also produces more interpretable and actionable profiles, K=3 was selected.

The three profiles

- Cluster 0 — Moderate Active, Steps Focused: Lightest group (74kg average), lowest fitness level, highest daily steps. They are active through movement rather than structured training.
- Cluster 1 — High Intensity Trainers: Heaviest effort score (128.92), longest sessions (89 minutes), most calories burned (24.58), highest heart rate (157 bpm). This group pushes hard when they show up.
- Cluster 2 — Consistent Low Intensity: Heaviest group (110kg), highest fitness level (12.83), most active days (76), longest streak (9.51 months). They are most consistent but train at the lowest intensity.

NB: Sleep, stress, and hydration were nearly identical across all three clusters. This suggests that for young adults aged 18 to 30, workout behavior and body composition drive behavioral differences more than daily lifestyle habits.

✓ Modeling - predicting Activity

```
print(df_model2['activity_type'].value_counts())
print(df_model2['activity_type'].dtype)
```

```
activity_type
Weight Training    18055
Swimming          17517
Running           16985
Name: count, dtype: int64
object
```

```
from sklearn.preprocessing import LabelEncoder
```

```
le = LabelEncoder()
df_model2['activity_encoded'] = le.fit_transform(df_model2['activity_type'])
print(dict(zip(le.classes_, le.transform(le.classes_))))
print(df_model2['activity_encoded'].value_counts())
```

```
{'Running': np.int64(0), 'Swimming': np.int64(1), 'Weight Training': np.int64(2)}
activity_encoded
2    18055
1    17517
0    16985
Name: count, dtype: int64
```

```
print(df_model2['cluster'].dtype)
print(df_model2['cluster'].value_counts())
```

```
int32
cluster
0    19684
2    16695
1    16178
Name: count, dtype: int64
```

```
# activity encoding
print("Activity encoding:")
print(dict(zip(le.classes_, le.transform(le.classes_))))
```

```
# cluster labels
print("\nCluster distribution:")
print(df_model2['cluster'].value_counts())
```

```
# what each cluster means
cluster_names = {0: 'Casual Movers', 1: 'High Intensity Trainers', 2: 'Casual Movers'}
print("\nCluster names:")
for k, v in cluster_names.items():
    print(f"Cluster {k} = {v}")
```

```
Activity encoding:
{'Running': np.int64(0), 'Swimming': np.int64(1), 'Weight Training': np.int64(2)}
```

```

Cluster distribution:
cluster
0      19684
2      16695
1      16178
Name: count, dtype: int64

Cluster names:
Cluster 0 = Casual Movers
Cluster 1 = High Intensity Trainers
Cluster 2 = Consistent Low Effort

```

```
# Baseline
```

```

from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# define features and target
features = ['age', 'bmi', 'weight_kg', 'height_cm', 'duration_minutes',
            'intensity', 'calories_burned', 'avg_heart_rate', 'hours_sle
            'stress_level', 'daily_steps', 'hydration_level', 'resting_h
            'blood_pressure_systolic', 'blood_pressure_diastolic', 'fitn
            'effort_score', 'total_active_days', 'active_months', 'longe
            'cluster']

X = df_model2[features]
y = df_model2['activity_encoded']

# 80/20 split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,

# baseline model
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

print(classification_report(y_test, y_pred_lr, target_names=['Running',

print(f"Baseline accuracy: {accuracy_score(y_test, y_pred_lr):.4f}")

```

	precision	recall	f1-score	support
Running	0.91	0.91	0.91	3432
Swimming	0.75	0.72	0.74	3555
Weight Training	0.79	0.82	0.80	3525
accuracy			0.82	10512
macro avg	0.82	0.82	0.82	10512
weighted avg	0.82	0.82	0.82	10512

Baseline accuracy: 0.8155
 /usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py
 STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in
<https://scikit-learn.org/stable/modules/preprocessing.html>
 Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-re
 n_iter_i = _check_optimize_result(

```
lr = LogisticRegression(max_iter=2000, random_state=42)
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)

print(classification_report(y_test, y_pred_lr, target_names=['Running',
print(f"Baseline accuracy: {accuracy_score(y_test, y_pred_lr):.4f}")
```

	precision	recall	f1-score	support
Running	0.92	0.93	0.93	3432
Swimming	0.79	0.75	0.77	3555
Weight Training	0.81	0.84	0.82	3525
accuracy			0.84	10512
macro avg	0.84	0.84	0.84	10512
weighted avg	0.84	0.84	0.84	10512

Baseline accuracy: 0.8393

Other models

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Decision Tree
dtt = DecisionTreeClassifier(random_state=42)
dtt.fit(X_train_scaled, y_train)
y_pred_dtt = dtt.predict(X_test_scaled)
```

```
# KNN
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
y_pred_knn = knn.predict(X_test_scaled)

# Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)

# Compare all
print(f"Logistic Regression: {accuracy_score(y_test, y_pred_lr):.4f}")
print(f"Decision Tree:      {accuracy_score(y_test, y_pred_dtc):.4f}")
print(f"KNN:                {accuracy_score(y_test, y_pred_knn):.4f}")
print(f"Random Forest:      {accuracy_score(y_test, y_pred_rf):.4f}")
```

```
Logistic Regression: 0.8393
Decision Tree:      0.9188
KNN:                0.5035
Random Forest:      0.8993
```

```
print(f"Train accuracy: {accuracy_score(y_train, dtc.predict(X_train_scaled)):.4f}")
print(f"Test accuracy:  {accuracy_score(y_test, y_pred_dtc):.4f}")
```

```
Train accuracy: 1.0000
Test accuracy:  0.9188
```

```
print(f"Train accuracy: {accuracy_score(y_train, rf.predict(X_train_scaled)):.4f}")
print(f"Test accuracy:  {accuracy_score(y_test, y_pred_rf):.4f}")
```

```
Train accuracy: 1.0000
Test accuracy:  0.8993
```

```
# New baseline
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, acc
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt
import seaborn as sns
```

```
activity_names = ['Running', 'Swimming', 'Weight Training']
```

```
models = {
    'Logistic Regression': lr,
    'Decision Tree': dtc,
```

```

'KNN': knn,
'Random Forest': rf
}

predictions = {
    'Logistic Regression': y_pred_lr,
    'Decision Tree': y_pred_dtc,
    'KNN': y_pred_knn,
    'Random Forest': y_pred_rf
}

# baseline comparison with cross validation
print("="*55)
print(f"{'Model':<25} {'CV Mean':>10} {'Test Acc':>10}")
print("="*55)
for name, model in models.items():
    cv = cross_val_score(model, X_train_scaled, y_train, cv=5)
    test_acc = accuracy_score(y_test, predictions[name])
    print(f"{'name':<25} {'cv.mean():>10.4f} {'test_acc:>10.4f}")
print("="*55)

```

```

=====
Model                  CV Mean    Test Acc
=====
Logistic Regression    0.8367    0.8393
Decision Tree          0.9147    0.9188
KNN                    0.4928    0.5035
Random Forest          0.8868    0.8993
=====

```

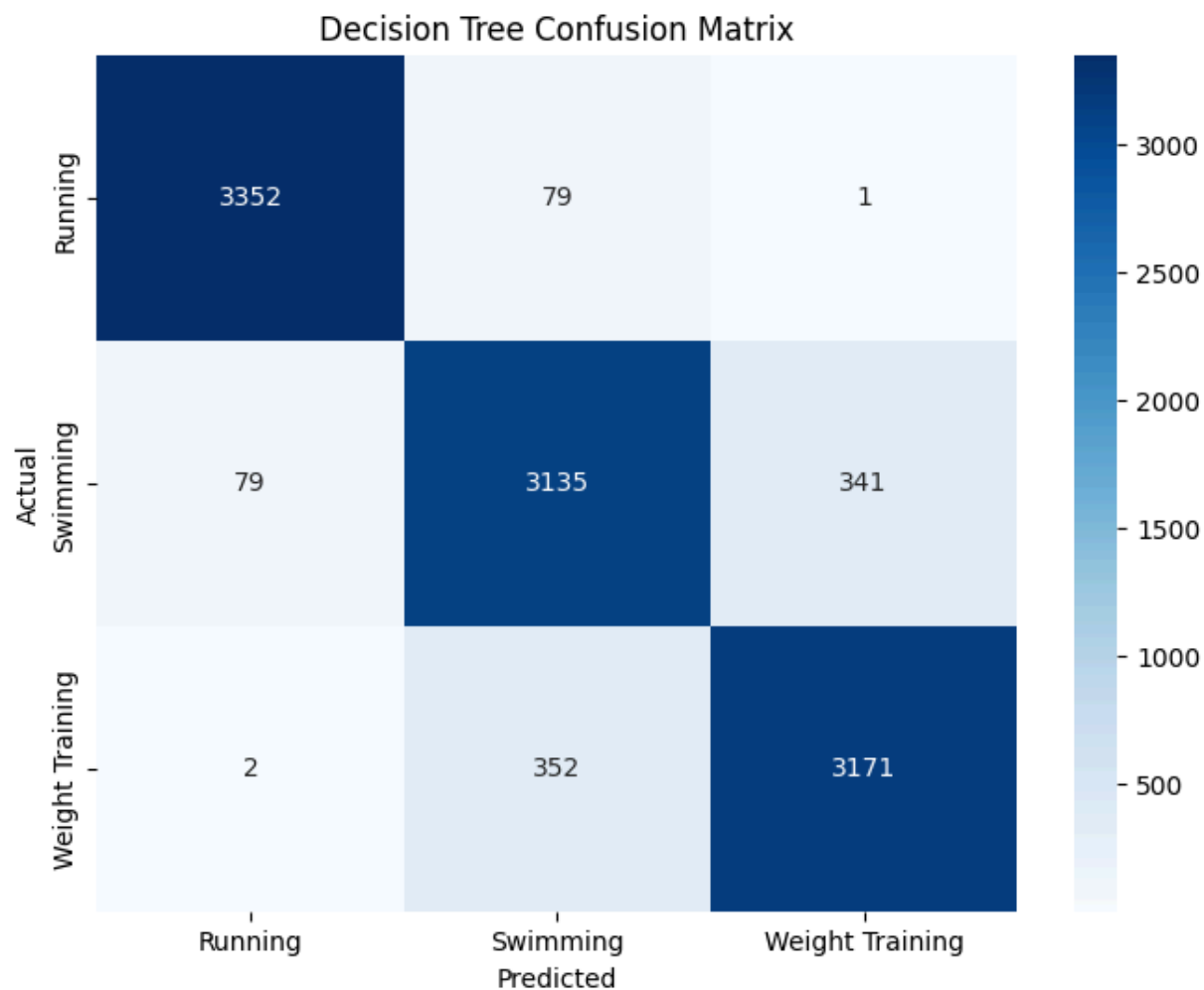
```

cm = confusion_matrix(y_test, y_pred_dtc)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=activity_names,
            yticklabels=activity_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Decision Tree Confusion Matrix')
plt.show()

print(classification_report(y_test, y_pred_dtc, target_names=activity_na

```



	precision	recall	f1-score	support
Running	0.98	0.98	0.98	3432
Swimming	0.88	0.88	0.88	3555
Weight Training	0.90	0.90	0.90	3525
accuracy			0.92	10512
macro avg	0.92	0.92	0.92	10512

```

from sklearn.metrics import accuracy_score

# Overall accuracy
acc = accuracy_score(y_test, y_pred_dtc)
misclass = 1 - acc

print(f"Accuracy:          {acc:.4f} ({acc*100:.2f}%)")
print(f"Misclassification: {misclass:.4f} ({misclass*100:.2f}%)")

```

```

Accuracy:          0.9188 (91.88%)
Misclassification: 0.0812 (8.12%)

```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

```

```
# Tuned Decision Tree Classifier
best_dtc = DecisionTreeClassifier(
    max_depth=20,
    min_samples_split=2,
    min_samples_leaf=10,
    max_leaf_nodes=50,
    random_state=42,
    max_features=None,
    splitter="best",
    criterion="gini"
)

best_dtc.fit(X_train_scaled, y_train)
y_pred_best = best_dtc.predict(X_test_scaled)

print(f"Tuned Decision Tree accuracy: {accuracy_score(y_test, y_pred_best)}")
print(classification_report(y_test, y_pred_best, target_names=activity_names))
```

```
Tuned Decision Tree accuracy: 0.6624
```

	precision	recall	f1-score	support
Running	0.82	0.85	0.83	3432
Swimming	0.54	0.48	0.51	3555
Weight Training	0.62	0.67	0.64	3525
accuracy			0.66	10512
macro avg	0.66	0.66	0.66	10512
weighted avg	0.66	0.66	0.66	10512

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

best_dtc = DecisionTreeClassifier(
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_leaf_nodes=None,
    random_state=42,
    max_features=None,
    splitter="best",
    criterion="gini"
)

best_dtc.fit(X_train_scaled, y_train)
y_pred_best = best_dtc.predict(X_test_scaled)

print(f"Tuned Decision Tree accuracy: {accuracy_score(y_test, y_pred_best)}")
```

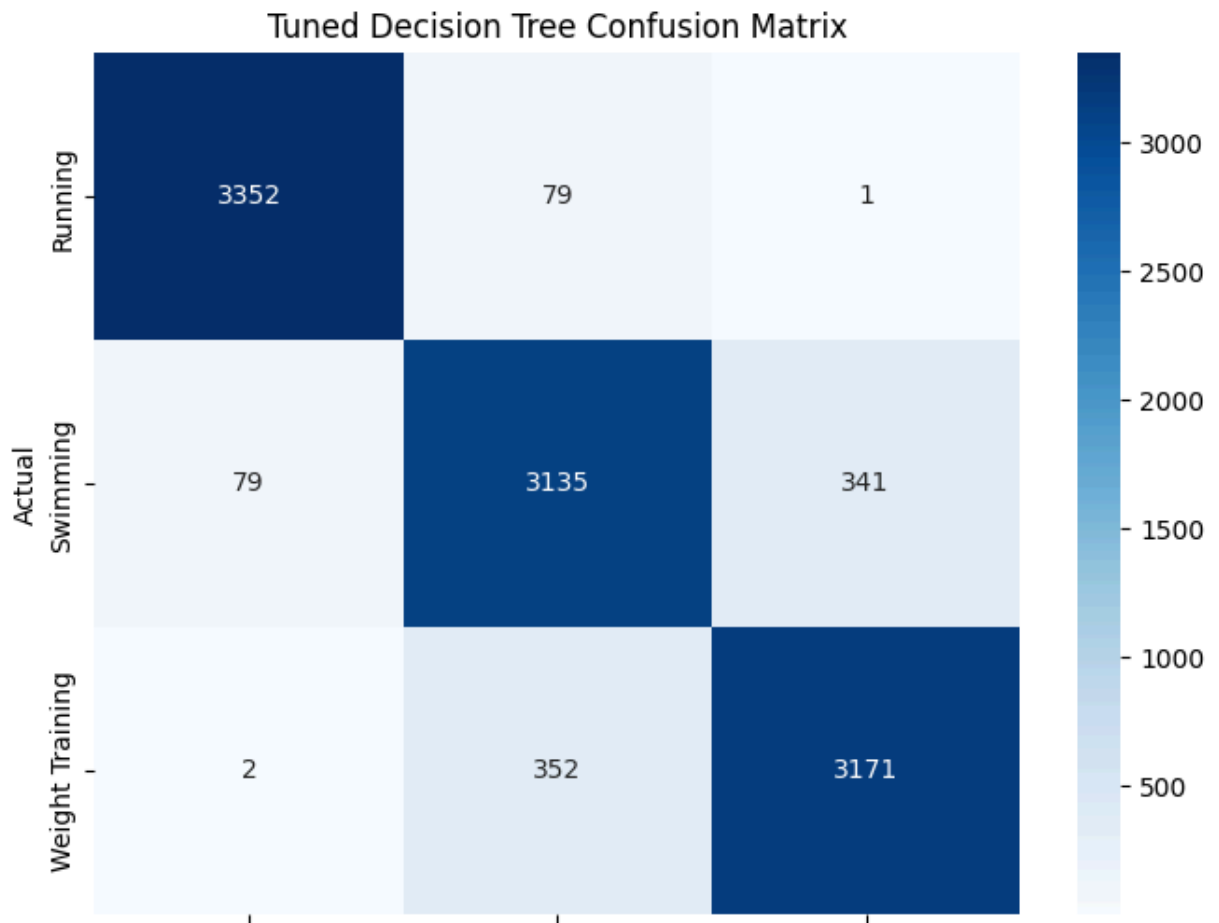
```
Tuned Decision Tree accuracy: 0.9188
```

```

cm = confusion_matrix(y_test, y_pred_best)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=activity_names,
             yticklabels=activity_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Tuned Decision Tree Confusion Matrix')
plt.show()

```



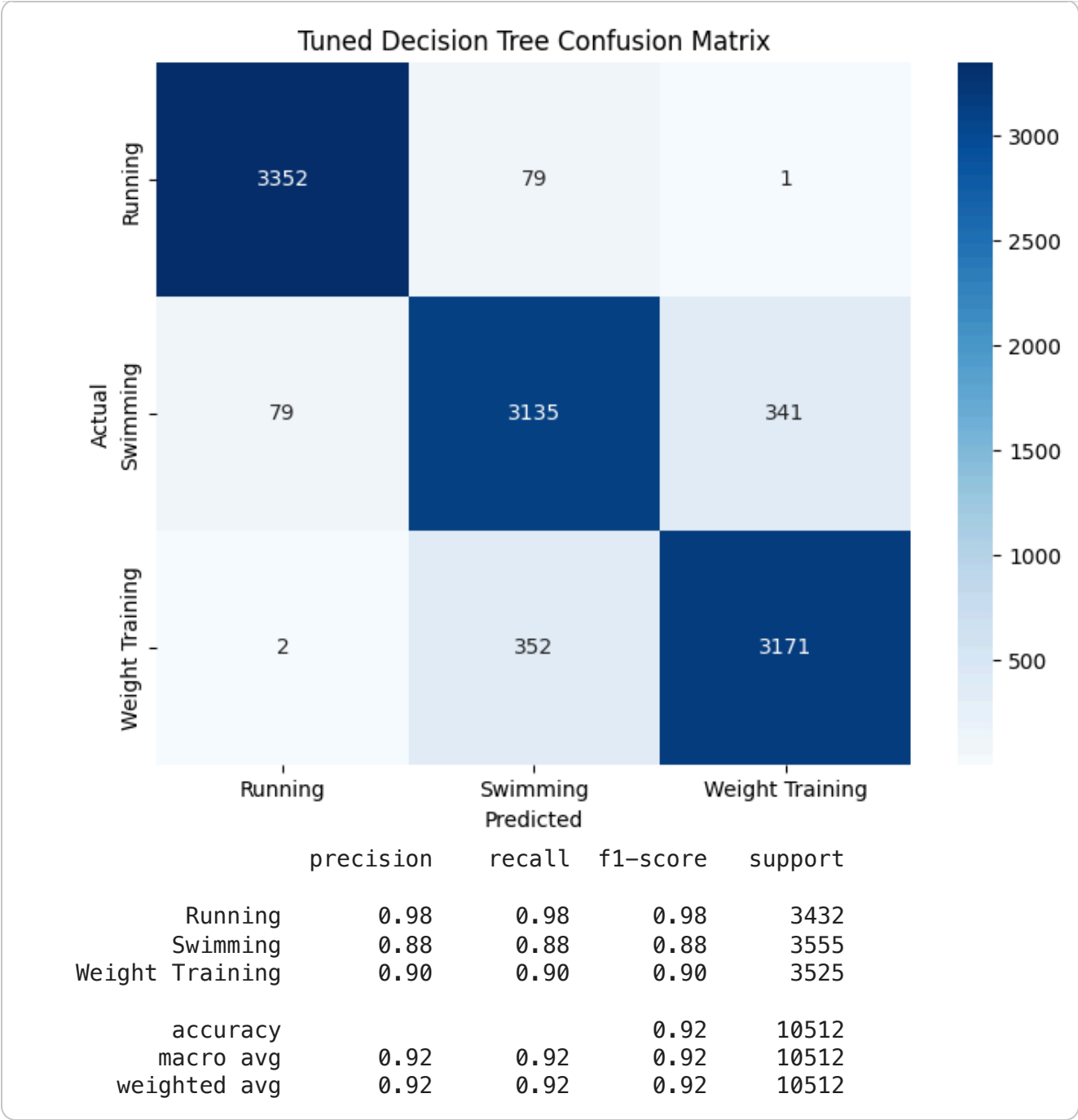
```

cm = confusion_matrix(y_test, y_pred_best)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
             xticklabels=activity_names,
             yticklabels=activity_names)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Tuned Decision Tree Confusion Matrix')
plt.show()

print(classification_report(y_test, y_pred_best, target_names=activity_n
print(f"Accuracy: {accuracy_score(y_test, y_pred_best):.4f}")
print(f"Misclassification: {1 - accuracy_score(y_test, y_pred_best):.4f}")

```



Modeling Summary

Baseline